3.性能

Shopify主题的最佳性能实践

构建或自定义主题时，一定要考虑性能，因为商家选择主题时，性能是重要因素。优化主题提高性能是增加用户体验的关键，性能直接影响转化率和搜索引擎排名。当主题提交到商店时，商店会进行性能评分，主题产品页，主页平均得分要在60以上。

性能优化

优化主题性能的最佳实践如下：

1. 优化JavaScript

使用以下原则优化性能的JavaScript:

- **减少JavaScript的使用**

  主要使用HTML和CSS构建主题，查找购买产品等基本功能不需要JavaScript,JavaScript应该使用在作为渐进性增强以及没有可用的HTML和CSS解决方案的情况下。

  CSS的解析和渲染速度比JavaScript快很多,所以应该使用CSS构建交互性。

  缩小后的 JavaScript 包大小应为 16 KB 以下。Shopify在店铺请求时自动缩小JavaScript。

- **减少对外部框架和库的依赖**

  如果需要使用JavaScript，请避免引入第三方框架、库和依赖，尽可能使用本地浏览器功能和DOM APIs。在软件包中包含JavaScript库可能会导致软件包太大、加载时间太慢等问题，不能有一个好的用户体验。React、Angular和Vue等框架，以及jQuery等大型实用程序库，都会带来巨大的性能成本。避免为非常旧的浏览器（任何不支持`async`(异步)/的浏览器`await`）引入polyfill库。如果你使用browserslist，那么你的目标浏览器的市场份额可以超过1%。

- **避免脚本阻塞页面的解析**

  脚本会阻塞DOM的构造和呈现，直到加载、解析和执行该脚本。它们还会造成网络拥塞，呈现延迟页面。这会影响第一个Contentful Paint和最大的Contentful Paint等指标。在脚本标记上使用`defer`(延迟)或`async`(异步)属性可以避免这种情况。标记为`async`的脚本不会阻止DOM构造或CSSOM，因为它们可以在构建CSSOM之前执行。使用`defer`,脚本直到页面加载事件触发后才执行，而`async`允许脚本在解析文档时在后台运行。

2. 关键资源预加载，延迟或避免加载其他资源

资源预加载允许浏览器在使用资源之前下载资源，预加载资源或者使用系统资源可以减少客户与页面交互之前需要下载的初始资源包的大小，

- **使用资源提示预加载关键资源**

  通过使用以下方法之一，每个模板最多可以向代码添加两个资源提示：

  - `preload_tag` 过滤器
  - `stylesheet_tag` 过滤器 `image_tag` 过滤器上的preload关键字参数

  当Shopify呈现带有预加载指令的页面时，它会在后续请求中发送一个预加载资源提示作为链接头。

- **延迟加载首屏图像**

  只要在页面需要时加载图像并且考虑页面滚动之前需要的占位符。因为页面看起来的加载速度比实际的加载速度快所以也有利于提高性能，使用 `image_tag: loading: 'lazy'`：

HTMLBars

```
{{ settings.favicon | image_url: width: 200 | image_tag: loading: 'lazy' }}
```

- **交互模式加载非关键资源**

  页面可能包含一些不经常使用的组件或资源，你可以使用交互模式加载这些资源，以避免加载、解析和执行不必要的代码。

- **使用系统字体**

  使用系统字体可以避免客户端在呈现文本之前需要下载其他资源。

3. 在Shopify服务器上托管资产

为您的资产使用相同的主机可以避免不必要的 HTTP 连接，并允许服务器使用 HTTP/2 优先级来优先传输阻塞资源。

4. 使用响应式图片

在手机上查看过大的图片可能不能正常显示，并且会减慢页面的加载速度，使用响应式图片会自动调整图片大小以适应设备屏幕。

指定图片大小可以确保在不降低质量的情况下，下载尽可能小的图片。这减少了对浏览器端缩放的依赖。

你可以使用 `image_tag` (使用智能默认宽度集返回图像的srcset)过滤器将响应式图片加到你的主题中，可以使用关键字 `sizes` 调整返回srcset的大小。

HTMLBars

```
{{ product.featured_image | image_url: width: 2000 | image_tag }}
//输入
```

HTML

```
<img
src="//cdn.shopify.com/s/files/1/0251/7476/9720/files/.png?v=1580676830&amp;wid
"
  alt=""
srcset="//cdn.shopify.com/s/files/1/0251/7476/9720/files/teal-polo.png?v=158067
 352w,
//cdn.shopify.com/s/files/1/0251/7476/9720/files/teal-polo.png?v=1580676830&amp
 832w,
//cdn.shopify.com/s/files/1/0251/7476/9720/files/teal-polo.png?v=1580676830&amp
 1200w,
//cdn.shopify.com/s/files/1/0251/7476/9720/files/teal-polo.png?v=1580676830&amp
 1920w"
  width="2000"
  height="2007">
  //输出
```

5. 优化Liquid代码

重复执行复杂的操作会增加Liquid的渲染时间，从而影响整体的存储速度。例如，你想按照价格对集合中的产品进行排序，你应该在遍历集合中的产品之前执行此操作，而不是作为循环代码的一部分，因为计算产品的顺序会增加请求的处理时间。

6. 使用主题检查来识别性能问题

主题检查可以识别以下内容：

- Liquid语法错误
- 缺少模版
- 未使用的变量和代码片段
- 未知或已弃用的标签
- 过多的代码片段嵌套
- 性能问题